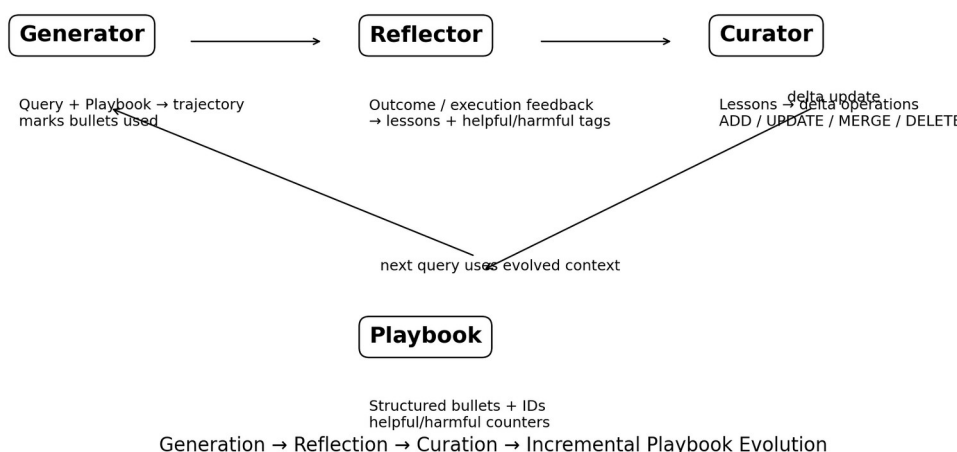


ACE

Agentic Context Engineering

Evolving Contexts for Self-Improving Language Models

从“管理当前窗口”到“跨任务累积可复用 know-how”的 Context Engineering



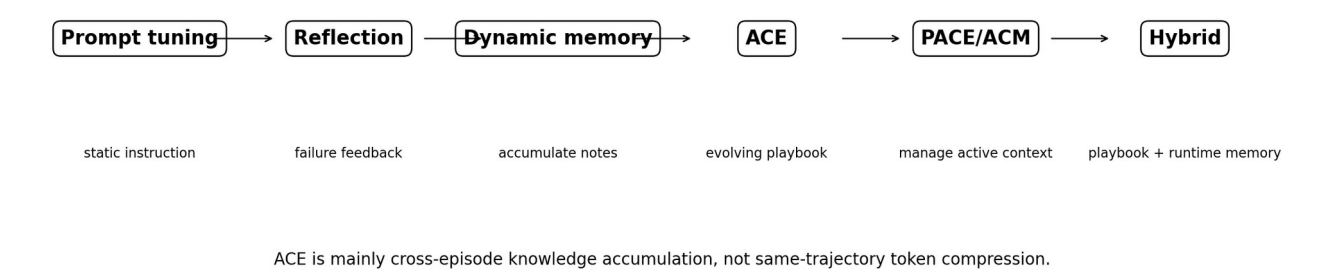
核心定位：ACE 不是长轨迹压缩器，而是一套让 Context 本身持续学习、增量生长、可解释更新的“非参数自改进”机制。

研究版本：ICLR 2026 / arXiv:2510.04618v3 (2026-03-29) | 官方实现：ace-agent/ace | 研究日期：2026-08-14

01. 一页结论：ACE 在 Context Engineering 谱系中的位置

先回答“它解决什么，而不是先讲它怎么做”。

前面一系列系统主要解决同一条 trajectory 内的上下文膨胀：ReSum 压缩历史，AgentFold 折叠多尺度状态，LongSeeker 让 Agent 主动编辑 working context，AdaCoM 学一个外部 context policy，ACM 提供可逆的短期/长期记忆，PACE 预测“下一步该读哪些历史”。ACE 解决的是另一条轴：一个 Agent 做完很多任务以后，如何把成功与失败沉淀为下一批任务可复用的上下文知识。



维度	ACE 的答案	为什么重要
学习对象	Context / Playbook，而非模型权重	部署快、可解释、可跨模型复用
时间尺度	跨 episode / 跨样本	把经验变成可积累 know-how
核心状态	结构化 bullet playbook	避免每轮重写造成 context collapse
更新方式	Generator → Reflector → Curator → Delta	把执行、反思、知识编辑解耦
增长策略	Grow-and-Refine	先积累细节，再去重/剪枝，而非先追求短

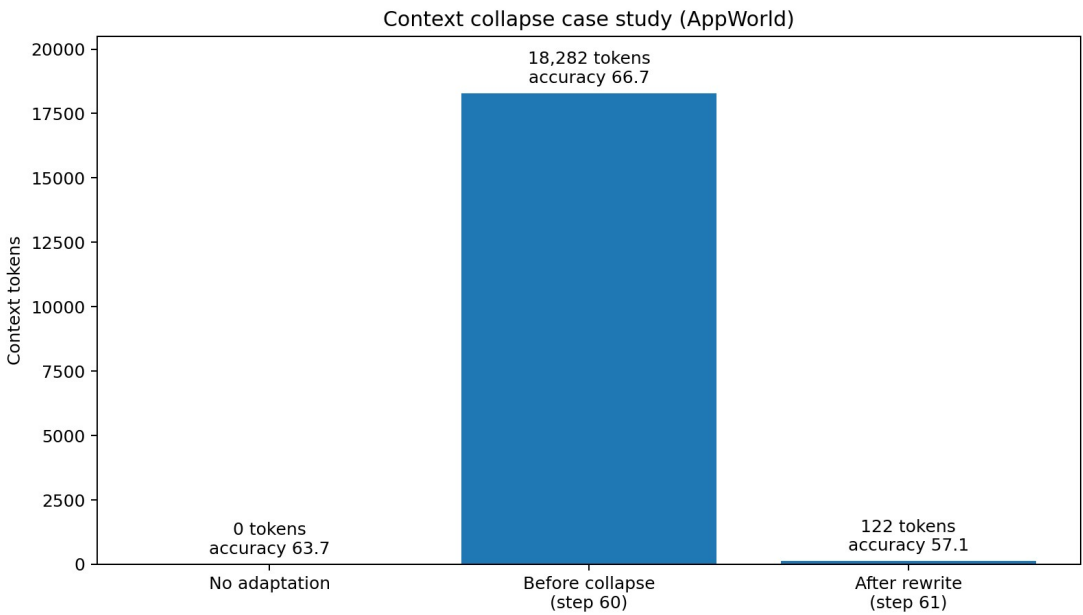
最值得迁移的思想：Runtime Context 与 Evolving Playbook 是两种不同的状态。前者服务“当前一步推理”，后者服务“跨任务学习”。生产 Agent 往往需要两者同时存在。

02. 为什么需要 ACE：Brevity Bias 与 Context Collapse

ACE 的 know-why 是反对“上下文越短越好”的默认假设。

传统 Prompt Optimization 常把目标隐式设成“找到更短、更通用的 instruction”。但对 Agent、工具调用、金融规则、API 约束等任务，真正有价值的往往是细节：哪个 API 有坑、哪个参数必须先校验、哪种失败模式意味着应该换路线。过早抽象会造成 brevity bias。

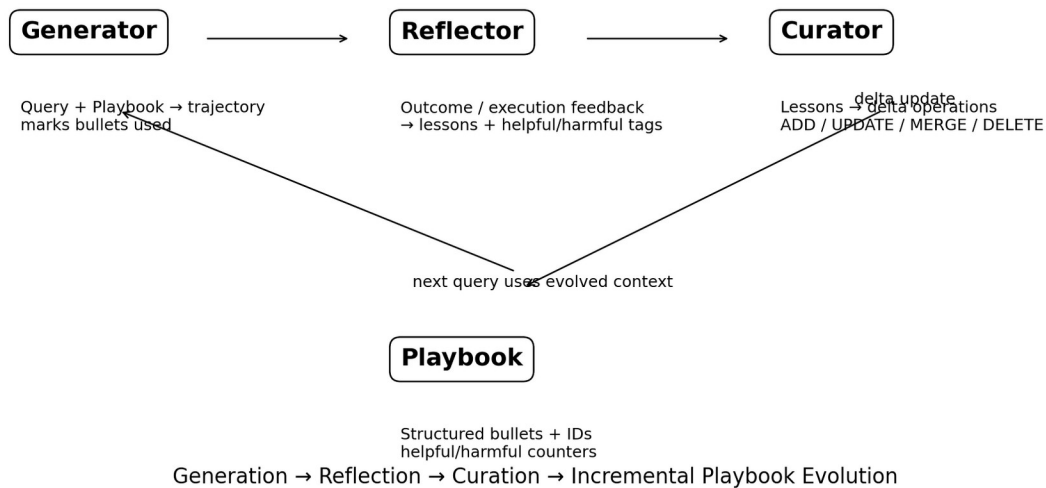
更严重的问题是 context collapse：如果每轮都要求 LLM 把整个旧 Context 重写成一个“更好的新 Context”，长时间迭代后模型可能突然把大量历史知识压成很短的总结。论文在 AppWorld 的案例中，Context 从 18,282 tokens 突然降到 122 tokens，准确率从 66.7 降到 57.1，甚至低于不做 adaptation 的 63.7。



失败模式	表面现象	根因	ACE 对策
Brevity bias	Prompt 越改越短	优化器偏好通用规则	允许详细 playbook 持续生长
Context collapse	一次 rewrite 丢掉大部分知识	LLM 全量重写具有高方差	只生成局部 delta
知识污染	错误经验被长期保留	反馈信号弱或错误	helpful/harmful 计数 + Reflector
无限增长	Context 越来越大	只 append 不 refinement	embedding 去重 + lazy pruning

03. 核心架构：Generator / Reflector / Curator

把“做任务”“解释结果”“编辑知识库”拆成三个角色。



Generator 读取 Query + 当前 Playbook，执行任务并产生 trajectory；同时显式标记自己使用了哪些 bullet ID。Reflector 读取 trajectory、预测结果、ground truth（如果有）或环境执行反馈，并判断哪些 bullet helpful / harmful / neutral，同时抽取新的策略性 insight。Curator 不直接重新写整个 Playbook，而是把 insight 转成 delta operations，再由确定性逻辑应用到旧 Playbook。

角色	输入	输出	工程职责
Generator	Query + Playbook + optional reflection	Answer / trajectory + bullet IDs	任务执行与经验暴露
Reflector	Trace + outcome + feedback + bullets used	Insights + utility tags	信用分配与错误归因
Curator	Current Playbook + reflection + budget	Delta operations	知识编辑与结构维护
Deterministic merger	Old playbook + operations	New playbook	减少 LLM rewrite 方差

关键 know-how：Reflector 和 Curator 分离。评价“发生了什么”与决定“知识库该怎么改”不是同一个问题，分开能显著降低单模型同时承担诊断、总结、编辑带来的认知负担。

04. Playbook 数据模型：把 Context 变成可操作的知识结构

不是一段 Prompt，而是一组有 ID、有反馈统计、有语义类别的最小知识单元。

STRATEGIES & INSIGHTS

[str-00001] helpful=5 harmful=0	Verify data types before processing	stable reusable strategy
[str-00002] helpful=3 harmful=1	Check edge cases before API calls	mixed evidence
[mis-00003] helpful=0 harmful=4	Avoid assuming tool success	failure-mode memory

ID enables reference; counters provide utility feedback; content preserves detailed know-how.

论文与仓库都把每条经验表示为一个 itemized bullet。官方 README 的格式类似 ``[str-00001] helpful=5 harmful=0 :: ...``。唯一 ID 让 Generator 可以引用；helpful/harmful counter 让 Reflector 的反馈能累积；Content 保存具体 strategy / domain concept / common failure mode。

字段	作用	设计收益
bullet_id	稳定引用与更新定位	可做细粒度 credit assignment
section	把策略、公式、错误模式分区	便于检索、审计、治理
helpful / harmful	累计使用反馈	把“这条经验好不好”变成状态
content	具体可复用策略	保留 domain-specific know-how

这意味着 ACE 的 Context 已经接近一个轻量的“可编辑知识库 IR”。它可以被检索、统计、局部更新、去重、过滤，而不是只能整段字符串替换。

生产启发：任何长期 Agent Memory 如果还只是一个不断 append 的 Markdown 文件，就缺少最关键的“稳定标识 + provenance + utility statistics + lifecycle operations”。

05. Incremental Delta Update：为什么局部编辑比整段重写稳定

ACE 最核心的 anti-collapse 机制。

ACE 不让 Curator 生成“完整的新 Playbook”，而是生成小规模 delta context。论文层面支持新增、修改、合并与删除；仓库 `Curator` 的 schema 也接受 ADD / UPDATE / MERGE / DELETE / CREATE_META。随后由非 LLM 逻辑做 merge。

这带来三个直接收益：Localization——只动相关知识；Fine-grained retrieval——可以只关注相关 bullet；Incremental adaptation——支持增量合并、去重与 pruning。更重要的是，它把一次高风险的全量“生成”变成低风险的局部“patch”。

Monolithic rewrite	Delta update
旧 Context 全部交给 LLM 重写	旧 Context 作为稳定基线
一次失败可能抹掉大量知识	错误通常局限在少数新增条目
难以追踪知识来自哪一步	每个 bullet 有 ID/计数/操作记录
长 Context 重写成本高	只生成少量 delta
难并行	多个 delta 可独立生成后 merge

可迁移抽象：把 Context Evolution 设计成 Git-style patching，而不是每次重新生成整个 repository。

源码注意：截至本次检查，`playbook\_utils.apply\_curator\_operations()` 中真正完整执行的主要还是 ADD；UPDATE / MERGE / DELETE / CREATE_META 在代码注释里仍标为 future operations。报告因此区分“论文框架能力”与“当前开源默认实现”。

06. Grow-and-Refine：为什么应该“先增长，再精炼”

ACE 反对一开始就为 Context 设过强的压缩目标。

新 insight 先以新 bullet 追加；已有 bullet 可以更新计数或内容；之后再通过语义 embedding 去重、合并或 pruning。Refinement 可以 proactive（每个 delta 后）或 lazy（接近 context window 时才触发）。

阶段	策略	目标
Grow	尽可能保留新策略、细节、失败模式	提高 recall，避免 brevity bias
Score	记录 helpful/harmful 使用证据	形成经验 utility
Refine	semantic dedup / merge / prune	控制冗余与噪声
Budget	到阈值再做更积极 pruning	控制 token / latency

论文的 sensitivity 分析很有工程价值：FiNER 上 max context length 从 10K、50K 到 100K，准确率分别约 78.6、78.4、78.3，说明 ACE 并不依赖精确调一个长度阈值；真正重要的是“不要在知识尚未积累前就压缩掉”。

Max context trigger	FiNER accuracy
10K	78.6
50K	78.4
100K	78.3

Know-why：对 LLM 来说，冗余通常是“成本问题”；丢失关键 domain detail 则是“能力问题”。ACE 优先解决能力，再用 refinement 解决成本。

07. Offline vs Online Adaptation：同一机制，两种学习时序

ACE 既可以当 Prompt Optimizer，也可以当 Test-Time Memory。

模式	何时更新 Playbook	反馈来源	典型用途
Offline	训练集上多轮 adaptation 后冻结	GT + execution feedback	系统 prompt / domain playbook 优化
Online	测试流中每个样本后继续更新	environment feedback，可无 GT	continual learning / agent memory
Offline warmup + Online	先离线建底座，再持续更新	混合	生产推荐形态

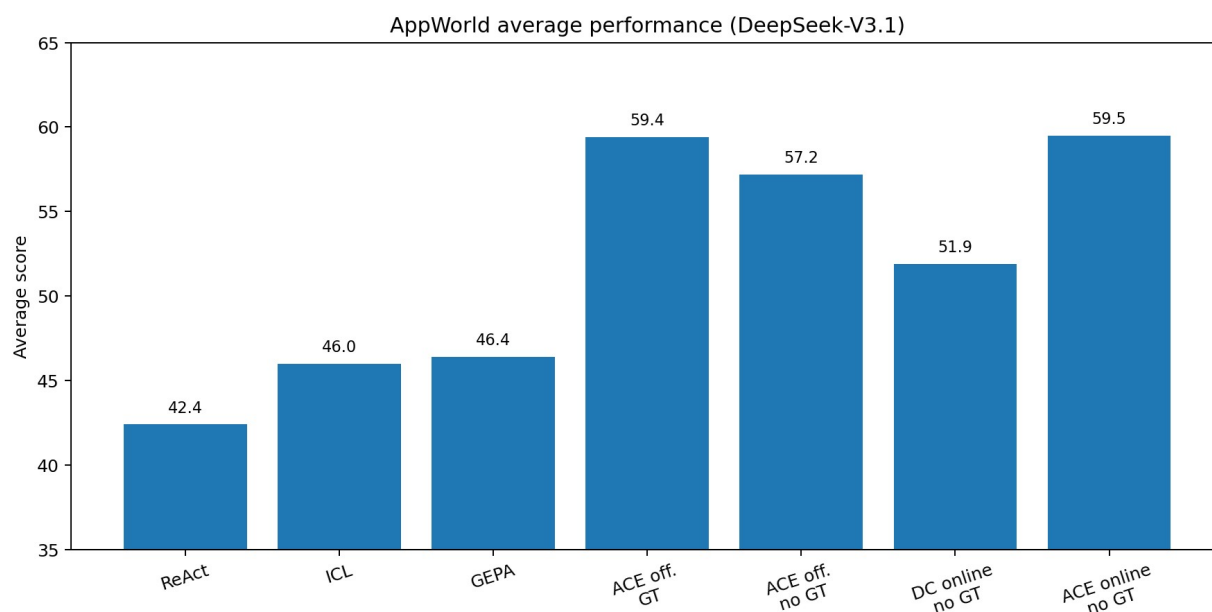
Offline adaptation 可以多 epoch 重放训练样本，逐步强化 Playbook。Online adaptation 则按顺序在 test stream 上：先用当前 context 做预测，再用该样本反馈更新 context。这个顺序非常重要，它保证评价没有“先看答案再做本题”的数据泄漏。

无 GT 能否工作取决于是否存在高质量 natural execution feedback。AppWorld 有 API / code execution 成功失败，因此 ACE 无 GT 仍显著提升；金融任务如果缺乏可靠外部信号，ACE 和 Dynamic Cheatsheet 都可能退化。

生产原则：能否做 Online Context Learning，不取决于“有没有标签”，而取决于“有没有足够可靠、可归因的反馈”。

08. AppWorld 实验：Context Evolution 对 Agent 的直接收益

基座：DeepSeek-V3.1-671B + 官方 ReAct。



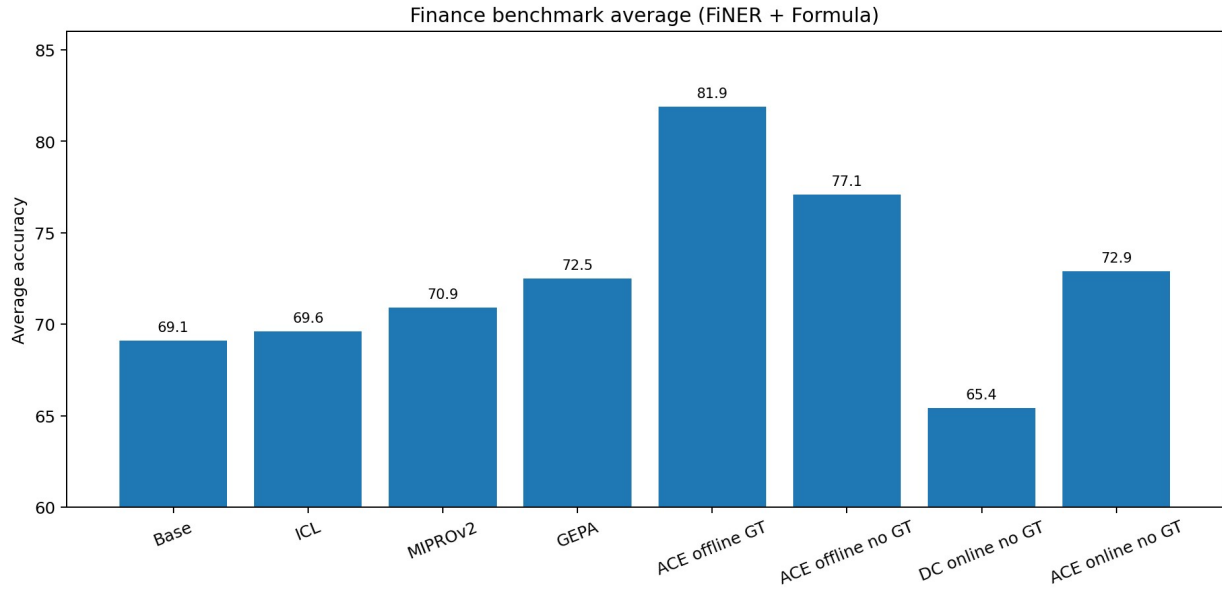
方法	设定	Average
ReAct	Base	42.4
ReAct + ICL	Offline	46.0
ReAct + GEPA	Offline	46.4
ReAct + ACE	Offline + GT	59.4
ReAct + ACE	Offline no GT	57.2
ReAct + DC	Online no GT	51.9
ReAct + ACE	Online no GT	59.5

Offline + GT 的 ACE 相比 ReAct 平均提升 17.0 points；无 GT 也提升 14.8 points。Online no-GT 场景相对 Dynamic Cheatsheet 平均领先约 7.6 points。论文还把 2025-09-20 的 AppWorld leaderboard 作为背景参照：59.4 已接近当时 production agent IBM CUGA 的 60.3，但作者明确说明这不是严格方法学基线。

实验说明了一个重要点：Agent 自改进不一定要先做 RL。若执行环境能给出有意义反馈，更新“上下文策略库”本身就能带来很大收益。

09. Finance / Domain 实验：为什么 Playbook 特别适合知识密集任务

FiNER + Formula 是 ACE 展示 detailed context 优势最明显的场景。



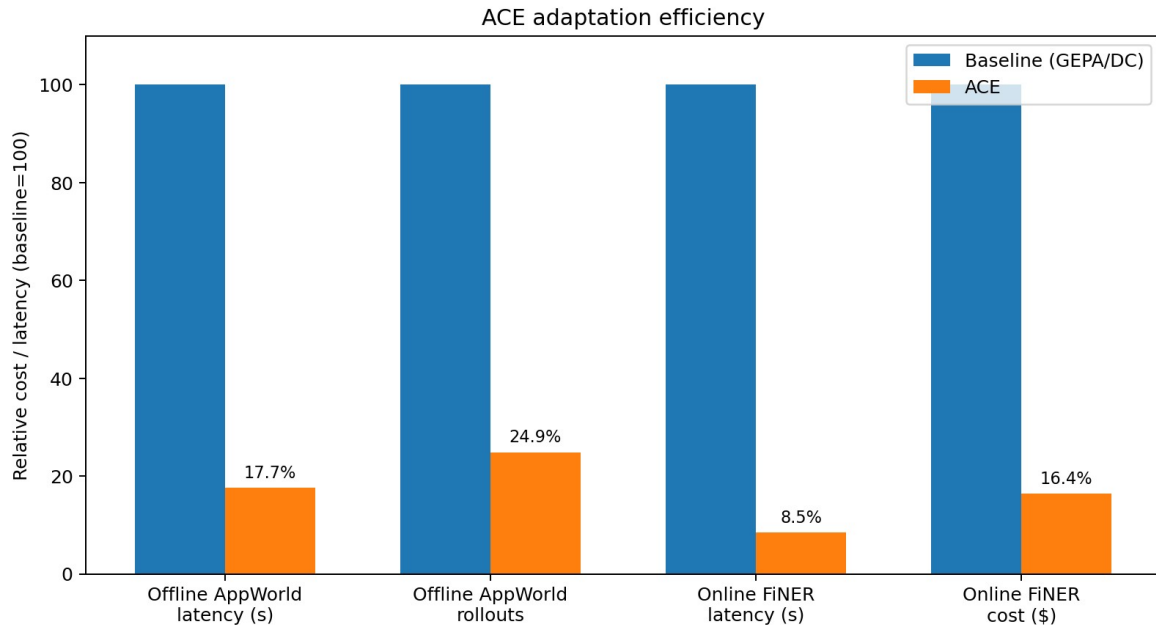
方法	FiNER	Formula	Average
Base	70.7	67.5	69.1
ICL offline	72.3	67.0	69.6
MIPROv2 offline	72.4	69.5	70.9
GEPA offline	73.5	71.5	72.5
ACE offline + GT	78.3	85.5	81.9
ACE offline no GT	71.1	83.0	77.1
DC online no GT	68.3	62.5	65.4
ACE online no GT	67.3	78.5	72.9

Formula 的提升尤其大，因为它需要累积具体金融概念、XBRL 规则与计算技巧。附录中 ACE 在 DDXPlus medical reasoning 上从 75.2 提升到 90.2；BIRD-SQL 平均从 47.8 提升到 52.9，说明 Playbook 机制并非只适用于 finance。

但 FiNER online no-GT 从 70.7 降到 67.3，正好暴露 ACE 的边界：如果任务没有可靠的 execution feedback，Reflector 可能把错误判断沉淀成长期知识。

10. 成本与速度：为什么长 Context 不必然意味着更贵

Delta update + 非 LLM merge + KV cache reuse 改变成本结构。



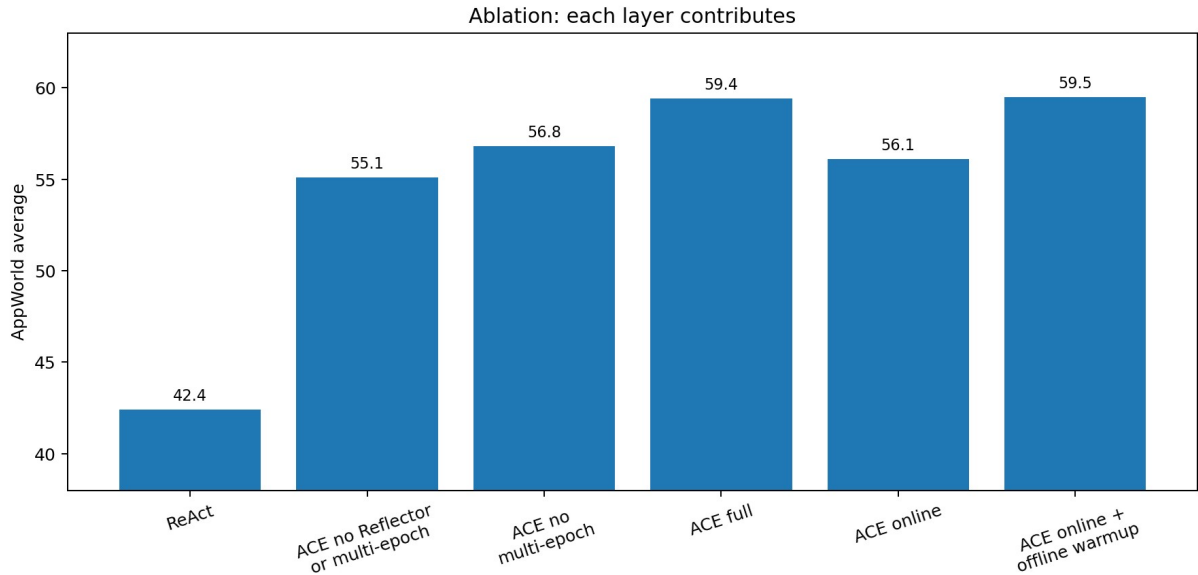
场景	Baseline	ACE	变化
Offline AppWorld latency	GEPA 53,898s	9,517s	-82.3%
Offline AppWorld rollouts	GEPA 1,434	357	-75.1%
Online FiNER latency	DC 65,104s	5,503s	-91.5%
Online FiNER token cost	DC \$17.7	\$2.9	-83.6%

论文进一步分析 OpenAI API (GPT-5.1) 的 prompt caching：ACE 在 evaluation 阶段约 91.8% input tokens 命中 cache，从而相对“按 raw context tokens 全额计费”减少约 82.6% 的 billed input-token cost。

系统层 know-how：长 Context 的成本要区分 raw tokens、prefill tokens、cache-hit tokens 和生成 tokens。只用“上下文长度”估算 serving cost 会误判具有高复用前缀的 Playbook 系统。

11. Ablation：真正有效的不是“三个 Agent 名字”，而是更新机制

论文通过 AppWorld 与附录 sensitivity 把关键因素拆开。



去掉 Reflector 与 multi-epoch 后，AppWorld 平均仍有 55.1；加入 Reflector 后到 56.8；完整 offline ACE 为 59.4。Online 直接学习为 56.1，而先做 offline warmup 再 online 达 59.5。

Ablation / sensitivity	结果	解释
No incremental update	Test-normal avg 56.9	仍受全量 rewrite 信息损失
With incremental update	Test-normal avg 70.3	Delta 是核心 anti-collapse 机制
Reflection 1 / 3 / 5 / 10 rounds	61.3 / 65.8 / 67.6 / 65.2	过少提取不足，过多会 overthink
Dedup 50% / 70% / 90%	77.0 / 73.9 / 78.6 FiNER	中等范围总体鲁棒
Harmful reflection every step	66.7 vs base 70.7	持续错误反馈会污染 playbook

核心结论：ACE 的性能来源主要是“结构化、增量、可追踪的知识更新”，而不是简单多调用几个 LLM。

12. 源码级实现：论文框架与当前仓库的真实边界

理解开源代码可以看出哪些设计已经 productionized，哪些仍是 conceptual。

源码模块	当前职责	值得注意的实现细节
ace/core/generator.py	生成答案并解析 bullet IDs	用 regex / JSON 提取被引用的 playbook item
ace/core/reflector.py	评估 trace 并打 helpful/harmful tag	支持有 GT / 无 GT 两套 prompt
ace/core/curator.py	把 reflection 转成 operations	schema 接受 ADD/UPDATE/MERGE/DELETE/CREATE_META
playbook_utils.py	解析 bullet、更新 counter、应用操作	当前 `apply_curator_operations` 主要完整实现 ADD
bulletpoint_analyzer.py	语义去重与 refinement	用于 grow-and-refine
ace/ace.py / ace_batch.py	offline / online orchestration	记录日志、intermediate playbook、评测

Generator 源码显式返回 `(full\_response, bullet\_ids\_used, call\_info)`；Reflector 把被用到的 bullet 标成 helpful / harmful / neutral；Curator 接收 token budget 与 playbook stats 后输出操作。这个数据流本质上已经形成一个“Context Control Plane”。

但是要注意 repository 当前操作执行层与论文抽象之间仍有 gap：Curator schema 虽允许 UPDATE/MERGE/DELETE 等，`apply\_curator\_operations` 的注释明确把多种操作标为 future operations。因此生产复现不能仅凭论文图假设所有编辑动作都已完整实现。

13. 反馈质量、Reflector 与 Context Poisoning

ACE 的上限由“你能否判断一次经验到底值不值得记住”决定。

论文显示较弱 Reflector 仍有收益：FiNER 中固定 Generator/Curator 为 DeepSeek-V3.1，Reflector 换成 GPT-OSS-120B 时准确率 76.6，DeepSeek-V3.1 为 78.3，GPT-5.1 为 78.5。这说明 ACE 不要求最强模型担任 Reflector。

更重要的是 harmful feedback stress test：每 5/10/25/50/100 次 adaptation 注入一次有害反思时，FiNER 仍分别保持 76.1/77.0/77.8/78.2/78.2；只有“每一步都注入有害反思”才跌到 66.7，低于 base 70.7。

风险	现象	推荐防护
False positive lesson	偶然成功被当成通用规则	要求多次证据 / confidence / provenance
Stale rule	API 或领域规则变化	加入时间戳、版本、失效机制
Contradictory bullets	多个经验互相冲突	contradiction detector + scoped applicability
Popularity bias	常用 bullet 因调用多而计数高	对 helpful count 做曝光归一化
Poisoning	错误反馈被长期积累	Verifier / trusted signals / quarantine

生产升级建议：helpful/harmful 不应该只是两个整数，而应升级为 evidence-weighted utility：按任务类型、模型版本、工具版本、反馈置信度分桶。

14. ACE vs PACE / ACM / AdaCoM / AgentFold

它们不是互斥方案，而是解决不同“时间尺度”的 Context 问题。

ACE vs same-trajectory context-management systems

System	Primary timescale	Main question	State form	Lossless?	Learned?
AgentFold	within trajectory	what history to fold?	multi-scale summaries	No	SFT
PACE	within trajectory	what history to read now?	multi-granularity chunks	Yes/External	No
AdaCoM	within trajectory	how should context be re-	message edits	depends	RL
ACM	within trajectory	when compress / recall?	short+long memory	Yes	OPD/KD
ACE	across episodes	what reusable know-how	evolving playbook bullets	Preserves bullets	LLM reflection

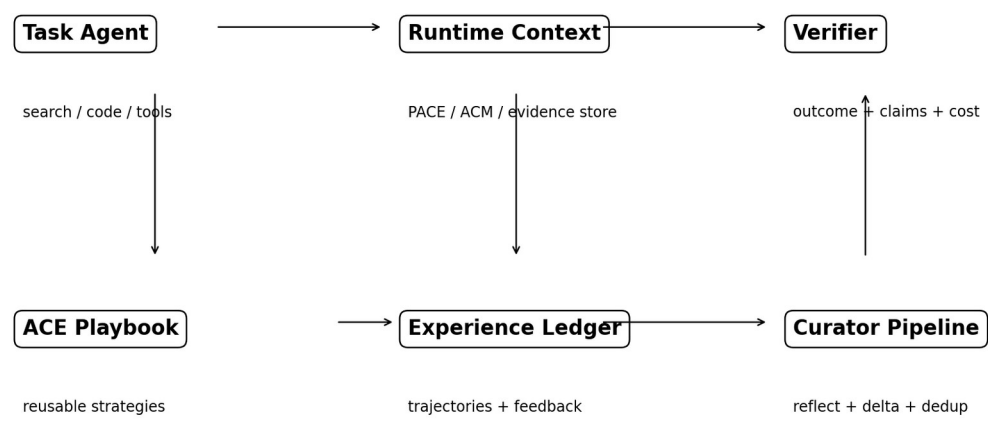
最容易混淆的是：PACE/ACM/AgentFold 主要回答“这一条长 trajectory 现在该保留哪些历史”；ACE 回答“做过很多 trajectory 以后，哪些经验应该成为以后任务的默认上下文”。因此 ACE 的 Playbook 更接近 policy prior / skill memory，而不是 working memory。

层级	典型系统	状态更新频率	目标
Within-step read policy	PACE	每一步	决定当前读取粒度
Within-trajectory memory	AgentFold / ACM	几十到几百步内	控制窗口、可恢复历史
External context controller	AdaCoM	每个 action 后	适配具体 Agent 的 context shape
Cross-episode knowledge	ACE	每个任务 / batch 后	累积 reusable know-how

因此最佳系统不是“选择 ACE 或 ACM”，而是 ACE × ACM/PACE：Playbook 提供长期策略先验；runtime context manager 控制当前一步真正进入模型的内容。

15. 生产级组合架构：Evolving Playbook × Runtime Context Manager

把“经验学习平面”与“即时推理平面”拆开。



Production synthesis: runtime context controls current tokens; ACE evolves cross-episode know-how.

平面	持久状态	核心接口	失败隔离
Task/Data Plane	working context + evidence	search/read/code/finish	单任务失败不污染长期知识
Runtime Context Plane	short/long memory	select/compress/query/recover	防 context overflow
Experience Plane	trajectory + feedback ledger	append event / score outcome	保留可审计原始经验
ACE Control Plane	versioned playbook	reflect / curate / publish	新 playbook 可 shadow / rollback
Verifier Plane	claims + execution + eval	score / contradiction / safety	控制长期记忆写入

推荐把 Playbook 更新做成异步 pipeline：在线 Agent 只写 experience ledger，不直接修改 production playbook；Reflector / Curator 在后台形成候选 delta，通过 regression eval 后发布新的 playbook version。这样既保留 ACE 的自改进能力，又避免即时错误反馈直接污染所有后续任务。

16. 可复用 Know-how：如果自己实现 ACE-like 系统

从论文机制提炼成工程检查表。

- 把“Prompt 字符串”升级为结构化 Context IR: stable ID、section、content、provenance、utility、scope、version。
- 任务执行模型只负责暴露使用了哪些知识项；不要让它同时负责编辑长期知识库。
- 把 outcome evaluation 与 knowledge curation 解耦：先判断因果与错误，再决定如何修改记忆。
- 所有长期更新都使用 delta / patch；禁止 LLM 无约束重写整个 playbook。
- 保留原始 trajectory 与 feedback ledger，确保每条长期规则都可追溯到证据。
- 先 Grow 后 Refine；不要以“最短 Context”为一等优化目标。
- 为每条经验记录适用范围：task family、tool/version、model family、time validity。
- 把 de-duplication、counter update、merge 等可确定操作放到 deterministic code，而不是全部交给 LLM。
- 发布新 Playbook 前做 shadow eval / canary / rollback；长期 Context 本质上也是生产配置。
- 结合 KV cache 与 prefix stability 设计 Playbook 布局，降低长 Context 的实际 serving 成本。

最重要的架构原则：Active Context 可以是有损 View；Experience Ledger 与 Published Playbook 的更新历史必须是可追踪、可回滚的。

17. 总结：ACE 把 Context Engineering 推向“非参数持续学习”

从 prompt engineering 到 context lifecycle engineering。

ACE 的意义不在于“又一个三 Agent workflow”，而在于重新定义了 Context：它不是一次 inference 的输入文本，而是一个长期演化、带结构、带反馈、带版本的策略状态。通过 Generator / Reflector / Curator 分工、incremental delta update 与 grow-and-refine，ACE 避免 brevity bias 与 context collapse，并在 AppWorld、Finance、Medical、Text-to-SQL 上展示了跨领域收益。

最终抽象：Self-Improving Agent $\approx$ Task Policy $\times$ Runtime Context Policy $\times$ Evolving Playbook $\times$ Experience Ledger $\times$ Verifier $\times$ Versioned Deployment

核心边界

ACE 不直接解决单条 500-step trajectory 的 token overflow；它应与 PACE / ACM / AgentFold 等 runtime context 系统结合。它也不是参数学习：知识主要存在于 Playbook 中，因此优点是可解释、快速、可跨模型，但缺点是依赖反馈质量并面临长期知识污染、冲突与版本治理问题。

主要参考资料

- [1] Zhang et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. ICLR 2026, arXiv:2510.04618v3.
- [2] Official ACE project page: ace-agent.github.io.
- [3] Official repository: github.com/ace-agent/ace (README, generator.py, reflector.py, curator.py, playbook_utils.py).
- [4] AppWorld benchmark protocol and official ReAct implementation, as cited/evaluated in the ACE paper.
- [5] Dynamic Cheatsheet, GEPA, MIPROv2 baselines, as described in the ACE paper.
- [6] Cross-series comparison uses the previously studied PACE, ACM, AdaCoM, AgentFold concepts as architectural reference; numerical claims in this report are ACE-paper-specific unless explicitly labeled otherwise.

术语澄清

2026 年另有一篇同样简称 ACE 的论文 “Pluggable Adaptive Context Elasticizer across Agents” (arXiv:2606.31564)。本报告中的 ACE 始终指 “Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models”，不要混用。